ReturnFlow: describe the return, not the form

Team: Zejun Xu, Peixuan Ji, Shuchen Liu **Midpoint check-in**

Repository: https://github.com/Zejun-Derry-Xu/DML_Project

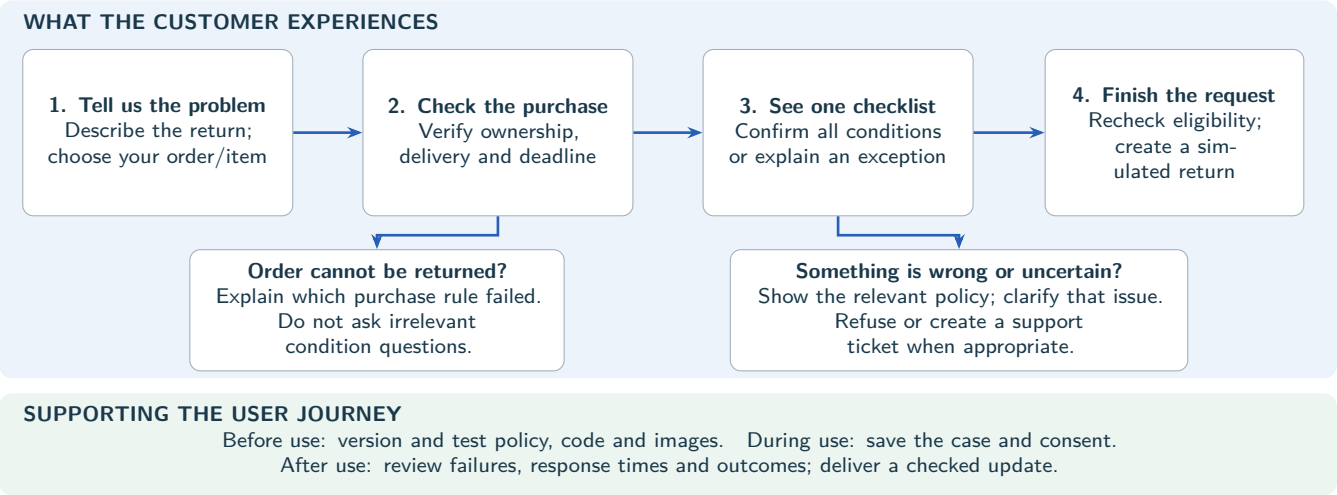
Jetstream2 demo entry (earlier prototype, verified October 24):

https://http-149-165-174-228-5173.proxy-js2-iu.exosphere.app/

This report describes our latest local checklist-led design with Qwen3 4B. The earlier guided prototype was accessible at the URL above. The VM is now shelved; the new release and its public URL still need cloud acceptance before submission.

1. Solution architecture: a conversation with a clear finish

A customer should be able to say what went wrong in their own words instead of learning the store's internal return form. ReturnFlow turns that description into candidate facts, checks the customer's order, and explains the conditions that matter. Because customers cannot be expected to volunteer every policy detail, the interface shows the remaining conditions together. They can answer them collectively or describe an exception; they do not need to complete a separate card for every condition.

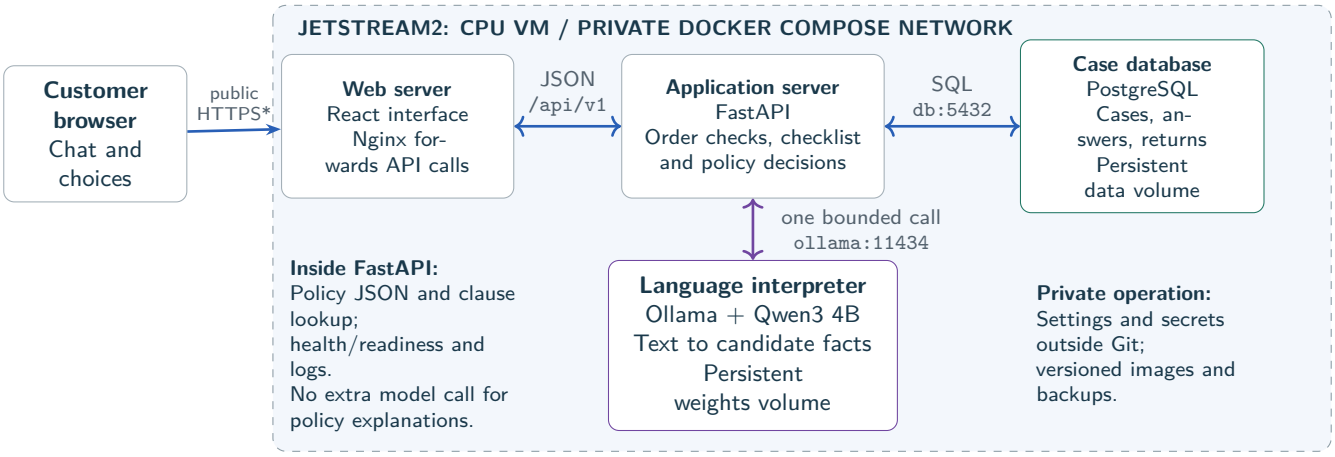


The normal path has a clear finish: after the order checks and checklist, an explicit *none of these problems apply, and submit* can supply the declaration and submission request together. Saying only *all clear* does not authorize submission. If tags were removed or a condition is uncertain, the system explains the relevant policy and asks only what is needed to resolve that exception. It never promises a real refund.

The same design covers the application's lifecycle. Developers maintain a versioned policy and test the application before promoting a release. During use, the frontend displays the conversation, owned-order choices and current checklist while the backend controls the next step. After use, request logs, health checks and latency metrics help the team find operational problems. PostgreSQL keeps the case, answers, checklist snapshot, consent, simulated returns and tickets across refreshes. Model weights live in a separate volume; code, policy and image versions are tracked in Git and the registry, while secrets and backups remain private.

2. Technical architecture: four services, separate responsibilities

We use one CPU-only Jetstream2 VM and Docker Compose rather than several agent services. The web server displays the interface, the application server runs the return workflow, PostgreSQL saves its state, and Ollama serves Qwen3 4B to interpret language. A policy file and clause search run *inside* the application server; they are not additional containers.



Blue: application requests and data. Purple: language interpretation.
The Exosphere proxy terminates HTTPS, then forwards HTTP to Nginx on VM port 5173.

A message travels from the browser through Nginx to `POST /api/v1/chat`. The server checks the account and selected order, then supplies the current message, selected item and relevant question to Qwen. The model returns structured candidates and supporting text, not a return decision. The backend checks that evidence occurs in the message, values have the expected types, and any collective declaration refers to the current checklist. It then applies policy rules, saves the case, and returns a reply and the next action. Button answers use `POST /api/v1/questions/{id}/answer`; refreshing reads the saved case rather than starting another return.

The policy decision uses trusted purchase data and a versioned configuration: delivery, the 30-day window, final-sale restrictions, duplicate checks and the declared condition requirements. Policy questions use lexical matching to retrieve and quote clauses from that same policy version; retrieval explains a decision rather than authorizing it. This small path needs no Qdrant. Only the web entry is public: the API binds to host loopback, and database/model ports are not published.

Delivery is deliberately simple. GitHub Actions checks Python, database migrations and API workflows, tests/builds the frontend, and smoke-tests the built application images with an isolated database. Approved triggers publish the same API/frontend image pair and a version manifest to GHCR. A human then promotes the release: back up the database, migrate, switch images and check readiness, version identity and a policy response. This is continuous delivery with manual promotion, not automatic production deployment. A guided image smoke does not qualify the real model. Separate volumes preserve database records and downloaded weights; rollback must retain compatible records and schema instead of deleting a volume.

3. Prototype: what the sentence means, and who extracts it

Our latest path uses **model-assisted field extraction**, not simply a list of keywords removed from a sentence. Qwen receives field definitions, examples and a JSON schema. It identifies the intent and copies a reason span or interprets an explicit condition. Program rules then validate that output and decide what to do. The examples below illustrate the contract, **not newly measured Qwen results**. Assume the customer is using a fictional account; condition replies refer to the selected item and current question.

Purple = meaning extracted by Qwen. Blue = action intent interpreted by Qwen.
Amber = exact format checks or deterministic application rules.

Example 1: preserve the customer's reason

> "This shirt is **too loose**. I **want to return it**."

The intended extraction is `reason_text="too loose"` and `intent="return"`. The prompt asks Qwen to copy the reason, not classify it into a fixed reason list; the customer's words remain available in the case. The program checks that the reason is an exact part of the quoted evidence and that the intent also has evidence. **A fit complaint does not establish stains, damage or tag condition.** The system therefore selects/checks the owned item and shows one conditions checklist, rather than approving immediately.

In this current path, both the request and the reason are interpreted by Qwen. It would be inaccurate to describe it as a regex catching *return* and passing only the remaining words to the model. The older rule extractor is retained for comparison, not as this production language pipeline.

Example 2: a format rule is not a condition rule

> "For **ORD-1001**, the **original tags are still attached**, but **I am not sure whether they are damaged**."

On intake, a narrow **ORD- followed by letters or digits** pattern can recognize an order reference when exactly one match is present. The backend still checks that the order belongs to the selected account; a matching identifier is not authorization. After item selection, Qwen interprets `tags_attached=true` and `tags_intact=null` with an uncertain state. The program requires genuine JSON booleans for definite conditions and null for uncertainty. It must not turn *not sure* into *undamaged*, or confuse tag damage with clothing damage. Simple controls such as an exact *cancel* are also handled directly by code.

Example 3: understand a declaration, then check permission

> "**None of those problems apply**, and **please submit my return**."

Qwen is asked to identify an overall `all_clear` response and a separate submission request, with the two evidence spans. **The program, not the model, binds that declaration to the displayed item, question and policy snapshot.** It rejects stale replies and does not let an overall declaration overwrite an earlier reported exception. If the final policy check passes, it creates one simulated return with duplicate protection. Without an explicit submission request, it asks for confirmation; if the model is unavailable, an explicitly labelled button can perform declaration and submission without pretending to understand free text.

Current limits and the next deployment step

The local candidate has automated workflow checks, but real-model interpretation can still be wrong even with valid JSON and a real quotation. It is not formally qualified, and the examples above are not an accuracy claim. We will deploy the selected 4B/checklist release only after its matching cloud checks, then update the public URL and verified status. The deployment template currently defaults to 8B, so the operator must explicitly select 4B. This remains a single-VM synthetic demo: no real refunds, production login or staffed support service is claimed.